Atividade de Reuso de Software

Mini CRM · Feature Model (FeatureIDE) · MPS.BR (GRU e DRU) · Database First

Aluno(a): _____

Disciplina: Desenvolvimento Orientado a Reuso de Software

Equipe: Lincoln Neto, Pedro Costa, Leandro Canha e Nicolas Hara

Projeto: MiniCRM-Reuso (Java 21 + Spring Boot 4, React 19, MySQL)

Branch da entrega: nicolas/reuso-gru-dru-featureide-dbfirst

Ferramentas: Eclipse IDE 2026-09 com FeatureIDE 3.12.0; MySQL 8.4; .NET 9 com EF Core (Database First)

Guia consultado: MPS.BR – Guia Geral MPS de Software:2016, páginas 36 (GRU) e 42 (DRU)

Parte I – Modelo de características do Mini CRM

1.1 Escopo e fontes

O Mini CRM é um sistema web para equipes de vendas: o vendedor cuida da própria carteira, registra interações, acompanha oportunidades em um funil e recebe lembretes; o gerente acompanha a equipe e o administrador controla o sistema. O modelo foi montado a partir de três fontes reais do projeto:

- **Documento de Requisitos do Mini CRM:** RF01 a RF33 e RNF01 a RNF09.
- **docs/specs (SPEC-00 a SPEC-04):** módulos, contratos e padrões de projeto da equipe.
- **Código do repositório:** enum Permissao (25 permissões), EtapaOportunidade, FaixaTarefa, EmailService/EmailServiceLog, AbstractExportadorService (CSV hoje, PDF previsto) e o schema MySQL com 9 tabelas.

O modelo representa a linha de produtos, ou seja, tudo o que o Mini CRM pode ser. O produto que a equipe construiu é apenas uma das configurações possíveis (seção 1.6): variantes que existem no modelo mas não no produto atual, como PostgreSQL, exportação em PDF ou e-mail SMTP, mostram a variabilidade que a linha de produtos comporta.

1.2 Execução no FeatureIDE (prints reais)

O Eclipse IDE 2026-09 e o FeatureIDE 3.12.0 foram instalados do zero. O projeto MiniCRM foi criado pelo assistente New → FeatureIDE Project (composer Feature Modeling), o modelo foi aberto no editor do FeatureIDE e a imagem foi gerada pela própria ferramenta, em Export As → Portable Network Graphics.

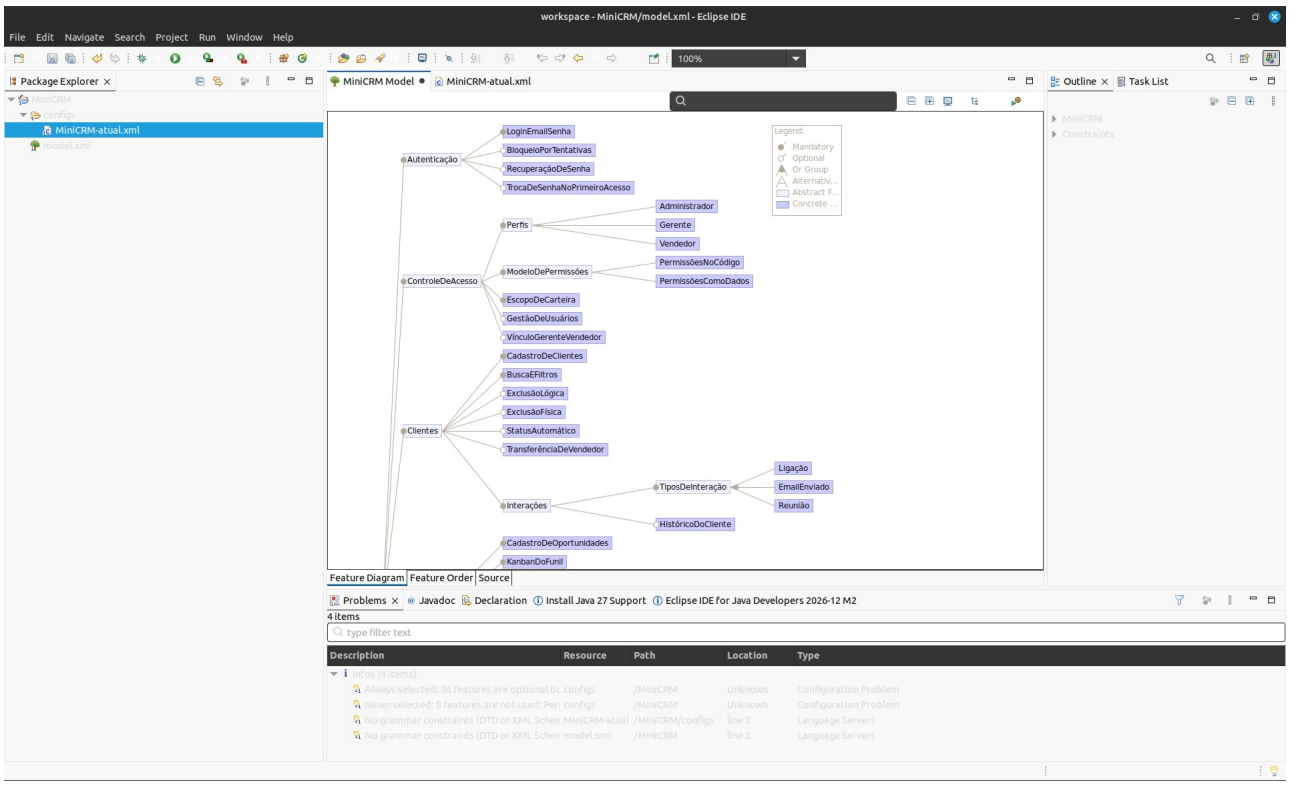


Figura 1 – Eclipse IDE com o FeatureIDE: projeto MiniCRM, editor do feature model (layout Left To Right) e painel Problems sem erros.

O diagrama completo exportado tem 74 características e 14 constraints. Por ser longo, as figuras 2 a 4 mostram recortes do mesmo arquivo PNG exportado pelo FeatureIDE (o arquivo inteiro está em docs/reuso/featureide/MiniCRM-FeatureModel.png); nada foi redesenhado.

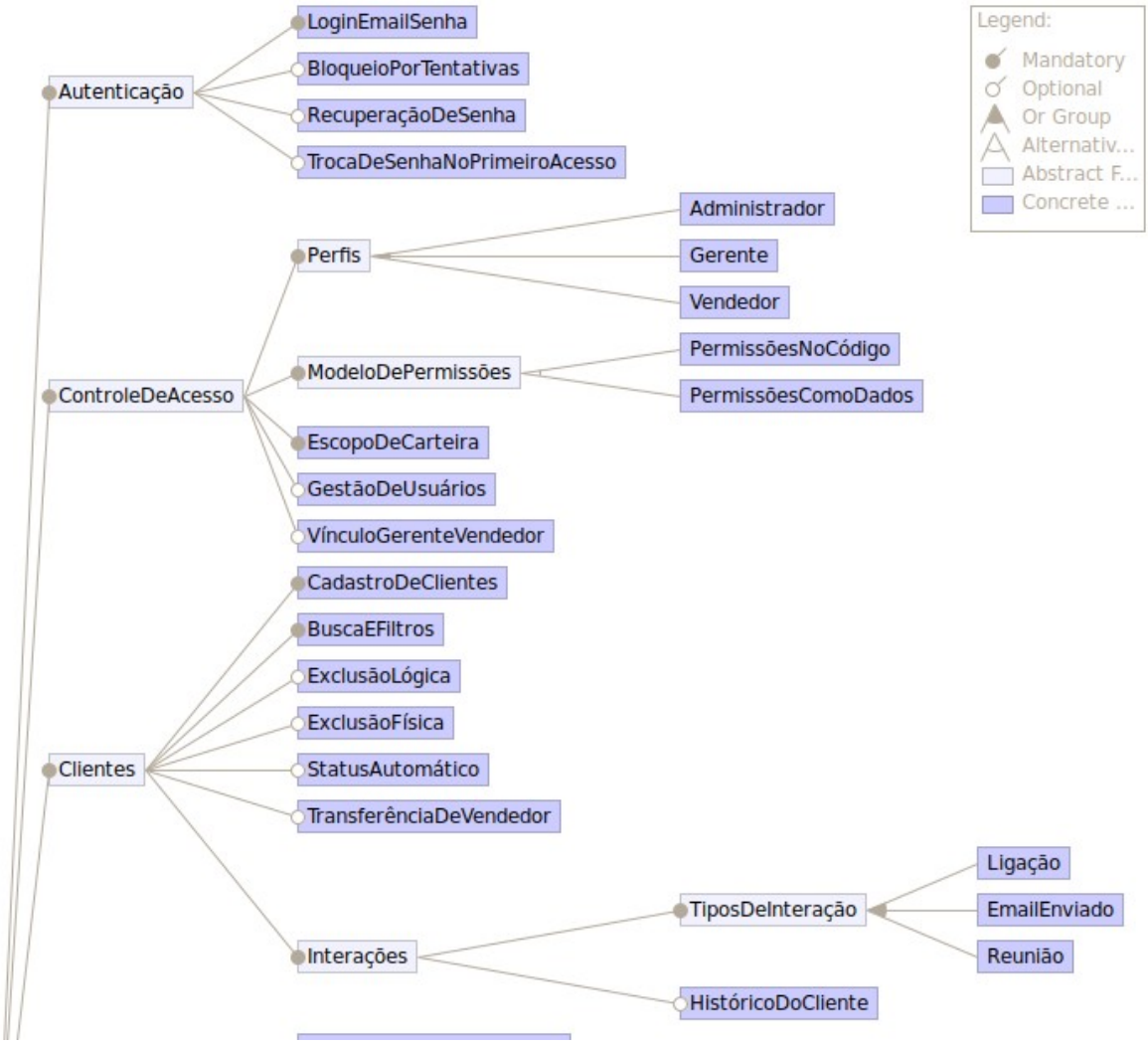


Figura 2 – Exportação do FeatureIDE (1/3): Autenticação, ControleDeAcesso e Clientes, com a legenda gerada pela ferramenta.

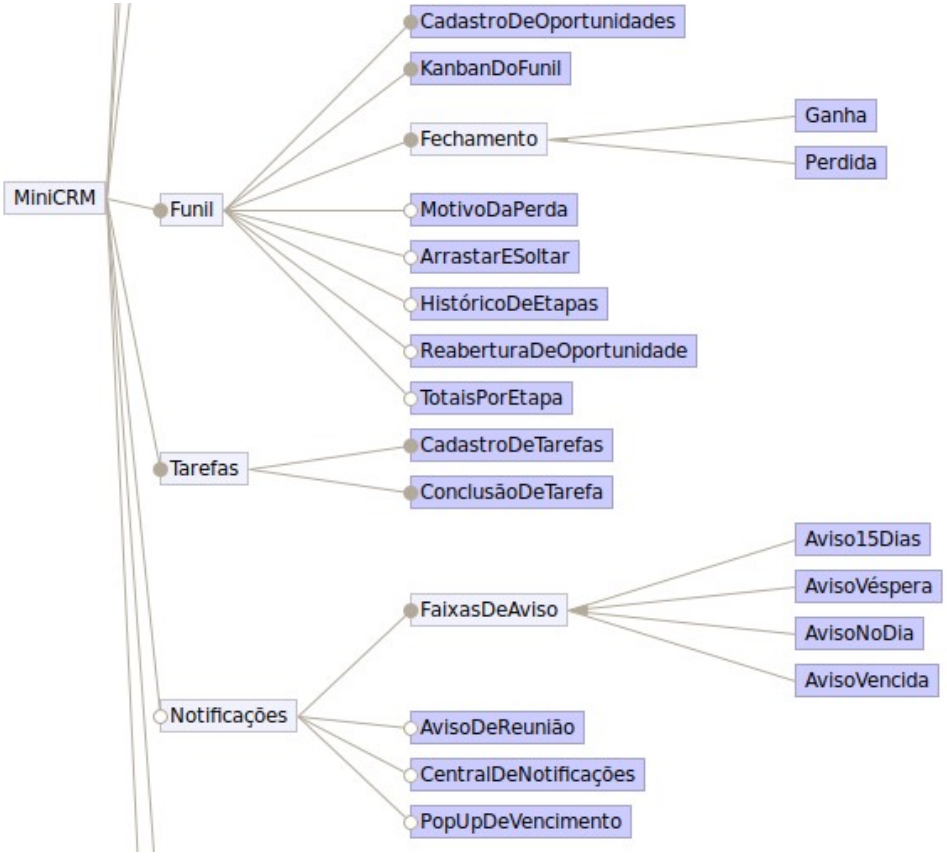
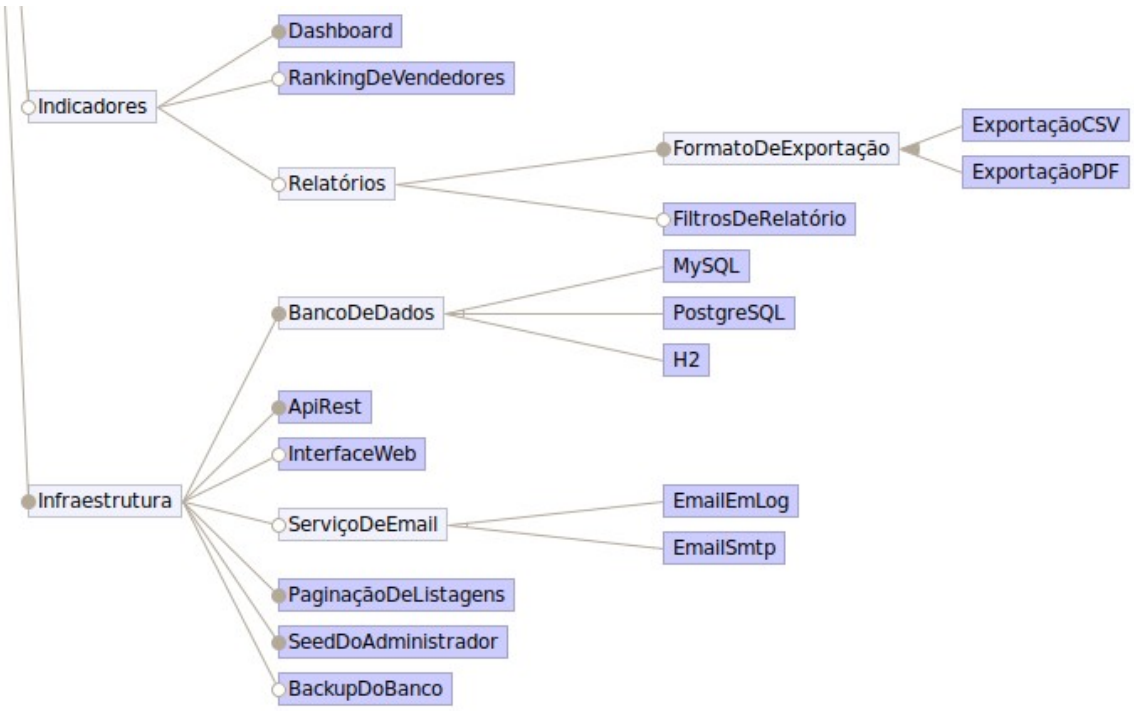


Figura 3 – Exportação do FeatureIDE (2/3): Funil, Tarefas e Notificações.



RecuperaçãoDeSenha ⇒ ServiçoDeEmail
MotivoDaPerda ⇒ Perdida
TransferênciaDeVendedor ⇒ Gerente ∨ Administrador
VínculoGerenteVendedor ⇒ Gerente ∧ Vendedor
RankingDeVendedores ⇒ Dashboard ∧ (Gerente ∨ Administrador)
Relatórios ⇒ Gerente ∨ Administrador
ArrastarESoltar ⇒ InterfaceWeb ∧ KanbanDoFunil
PopUpDeVencimento ⇒ InterfaceWeb
GestãoDeUsuários ⇒ Administrador ∨ Gerente
PermissõesComoDados ⇒ GestãoDeUsuários
¬(ExclusãoLógica ∧ ExclusãoFísica)
BackupDoBanco ⇒ ¬H2
StatusAutomático ⇒ Ganha
AvisoDeReunião ⇒ Reunião

Figura 4 – Exportação do FeatureIDE (3/3): Indicadores, Infraestrutura e as 14 constraints.

1.3 Catálogo das 74 características

O modelo tem 74 características: 19 abstratas e 55 concretas. A coluna “Relação” diz como a característica se liga ao pai no diagrama; “Fonte” indica o requisito (RF/RNF) ou a spec de onde ela veio.

#	Característica	Tipo	Relação	Fonte	Descrição
1	MiniCRM	Abstract	Raiz	—	Linha de produtos Mini CRM (raiz).
2	Autenticação	Abstract	Mandatory	RF01, RF17, RF32	Identificação e acesso do usuário.
3	LoginEmailSenha	Concrete	Mandatory	RF01, RNF03	Login com e-mail e senha, token JWT e senha com hash BCrypt.
4	BloqueioPorTentativas	Concrete	Optional	RF01	Bloqueio da conta após 5 tentativas erradas.
5	RecuperaçãoDeSenha	Concrete	Optional	RF17	Código de verificação por e-mail (1 h, uso único).
6	TrocaDeSenhaNoPrimeiroAcesso	Concrete	Optional	RF32	Senha provisória com troca obrigatória no primeiro acesso.

#	Característica	Tipo	Relação	Fonte	Descrição
7	ControleDeAcesso	Abstract	Mandatory	RF02, RF14, RF25, RF27	Perfis, permissões e visibilidade dos dados.
8	Perfis	Abstract	Mandatory	RF02	Perfis de acesso suportados pelo produto.
9	Administrador	Concrete	Or Group	RF02	Controla o sistema inteiro.
10	Gerente	Concrete	Or Group	RF02, RF27	Acompanha a equipe e cria vendedores.
11	Vendedor	Concrete	Or Group	RF02	Cuida da própria carteira.
12	ModeloDePermissões	Abstract	Mandatory	RF02, RNF03	Onde as permissões por ação são definidas.
13	PermissõesNoCódigo	Concrete	Alternative Group	RF02	Permissões por perfil fixas no código (enum Permissoes).
14	PermissõesComoDados	Concrete	Alternative Group	RF25	Permissões em tabelas do banco, editáveis em tela.
15	EscopoDeCarteira	Concrete	Mandatory	RF15, RF26	Cada usuário enxerga só a própria carteira; o gerente, a da equipe.
16	GestãoDeUsuários	Concrete	Optional	RF14, RF20, RF25	Cadastro, edição, desativação e reativação de usuários.
17	VínculoGerenteVendedor	Concrete	Optional	RF27	Vendedor fica vinculado ao gerente que o criou.
18	Clientes	Abstract	Mandatory	RF03 a RF06, RF26, RF30	Carteira de clientes e relacionamento.
19	CadastroDeClientes	Concrete	Mandatory	RF03	Criar, visualizar e editar clientes.
20	BuscaEFiltros	Concrete	Mandatory	RF04	Busca por nome/empresa e filtros por status e vendedor.
21	ExclusãoLógica	Concrete	Optional	RF03	Cliente excluído vira inativo e preserva o histórico.
22	ExclusãoFísica	Concrete	Optional	—	Variante de linha de produtos: remoção definitiva do registro (não usada hoje).
23	StatusAutomático	Concrete	Optional	RF30	Cliente vira ATIVO na primeira oportunidade ganha.
24	TransferênciaDeVendedor	Concrete	Optional	RF26	Troca do vendedor responsável pelo cliente.
25	Interações	Abstract	Mandatory	RF05, RF06	Registro de contatos com o cliente.
26	TiposDeInteração	Abstract	Mandatory	RF05	Canais de contato registráveis.
27	Ligação	Concrete	Or Group	RF05	Interação por ligação telefônica.
28	EmailEnviado	Concrete	Or Group	RF05	Interação por e-mail.
29	Reunião	Concrete	Or Group	RF05, RF23	Interação por reunião (pode gerar aviso).
30	HistóricoDoCliente	Concrete	Optional	RF06, RF16	Linha do tempo com interações, oportunidades e tarefas.
31	Funil	Abstract	Mandatory	RF07 a RF09, RF19, RF29	Funil de vendas e oportunidades.
32	CadastroDeOportunidades	Concrete	Mandatory	RF07	Criar oportunidades ligadas a um cliente.

#	Característica	Tipo	Relação	Fonte	Descrição
33	KanbanDoFunil	Concrete	Mandatory	RF08, RF09	Quadro com as etapas Prospecção, Contato, Proposta e Fechado.
34	Fechamento	Abstract	Mandatory	RF08	Desfechos possíveis de uma oportunidade.
35	Ganha	Concrete	Or Group	RF08	Venda concluída.
36	Perdida	Concrete	Or Group	RF08	Venda perdida.
37	MotivoDaPerda	Concrete	Optional	RF29	Motivo obrigatório ao perder (preço, concorrente, sem verba, sem resposta).
38	ArrastarESoltar	Concrete	Optional	RNF09	Mover cards entre etapas arrastando.
39	HistóricoDeEtapas	Concrete	Optional	RF09	Registro de cada mudança de etapa.
40	ReaberturaDeOportunidade	Concrete	Optional	RF09	Reabrir oportunidade fechada.
41	TotaisPorEtapa	Concrete	Optional	RF19	Contagem e soma de valores por etapa.
42	Tarefas	Abstract	Mandatory	RF10, RF28	Tarefas e lembretes da equipe.
43	CadastroDeTarefas	Concrete	Mandatory	RF10	Tarefas com vencimento ligadas a clientes e oportunidades.
44	ConclusãoDeTarefa	Concrete	Mandatory	RF28	Marcar a tarefa como concluída.
45	Notificações	Abstract	Optional	RF13, RF23, RF24	Avisos internos ao usuário.
46	FaixasDeAviso	Abstract	Mandatory	RF13	Faixas de vencimento que geram aviso.
47	Aviso15Dias	Concrete	Or Group	RF13	Aviso 15 dias antes do vencimento.
48	AvisoVéspera	Concrete	Or Group	RF13	Aviso na véspera.
49	AvisoNoDia	Concrete	Or Group	RF13	Aviso no dia do vencimento.
50	AvisoVencida	Concrete	Or Group	RF13	Aviso de tarefa vencida.
51	AvisoDeReunião	Concrete	Optional	RF23	Notificação de reuniões agendadas.
52	CentralDeNotificações	Concrete	Optional	RF23	Aba com lida/não lida e contador.
53	PopUpDeVencimento	Concrete	Optional	RF24	Pop-up de aviso no canto da tela.
54	Indicadores	Abstract	Optional	RF11, RF12, RF15, RF31	Dashboard, ranking e relatórios.
55	Dashboard	Concrete	Mandatory	RF11, RF15	Indicadores por perfil: clientes, oportunidades e tarefas.
56	RankingDeVendedores	Concrete	Optional	RF31	Ranking por valor ganho e oportunidades fechadas.
57	Relatórios	Abstract	Optional	RF12, RF21, RF22	Exportação de listas.
58	FormatoDeExportação	Abstract	Mandatory	RF12	Formatos de arquivo exportáveis.
59	ExportaçãoCSV	Concrete	Or Group	RF12	CSV com BOM, separador ; e escape RFC 4180.
60	ExportaçãoPDF	Concrete	Or Group	RF12	PDF (desejável no requisito).
61	FiltrosDeRelatório	Concrete	Optional	RF21	Filtros por período e status antes de exportar.
62	Infraestrutura	Abstract	Mandatory	RNF03 a RNF06, RF18, RF32	Plataforma técnica do produto.

#	Característica	Tipo	Relação	Fonte	Descrição
63	BancoDeDados	Abstract	Mandatory	RNF06	SGBD usado pelo produto.
64	MySQL	Concrete	Alternative Group	RNF06	MySQL 8 (configuração atual).
65	PostgreSQL	Concrete	Alternative Group	—	PostgreSQL.
66	H2	Concrete	Alternative Group	—	H2 em memória (desenvolvimento e testes).
67	ApiRest	Concrete	Mandatory	SPEC-00	API REST versionada em /api/v1.
68	InterfaceWeb	Concrete	Optional	RNF01, RNF04, RNF08	Frontend web React.
69	ServiçoDeEmail	Abstract	Optional	RF17	Implementação de envio de e-mail.
70	EmailEmLog	Concrete	Alternative Group	RF17	Adapter que só registra o e-mail em log (atual).
71	EmailSmtp	Concrete	Alternative Group	RF17	Envio real por SMTP (previsto).
72	PaginaçãoDeListagens	Concrete	Mandatory	RF18	Listagens paginadas em 20 registros.
73	SeedDoAdministrador	Concrete	Mandatory	RF32	Administrador inicial criado na primeira execução.
74	BackupDoBanco	Concrete	Optional	RNF06	Backup semanal do banco.

1.4 Justificativas por conceito

Cada conceito do FeatureIDE foi aplicado onde ele representa uma regra real do Mini CRM, e a função de cada um está resumida na tabela a seguir.

Conceito	Função no modelo	Qtde	Onde aparece
Abstract	Organizar o modelo em módulos e pontos de variação; não vira código sozinha.	19	Raiz, 8 módulos e os grupos (Perfis, ModeloDePermissões, TiposDeInteração…)
Concrete	Representar funcionalidades e escolhas que viram código ou configuração.	55	Todas as folhas do diagrama
Mandatory	Garantir o núcleo presente em todo produto.	26	LoginEmailSenha, CadastroDeClientes, KanbanDoFunil, Dashboard, ApiRest…
Optional	Marcar diferenciais que o produto pode ter ou não.	26	BloqueioPorTentativas, MotivoDaPerda, RankingDeVendedores, BackupDoBanco…
Or Group	Permitir uma ou mais opções combináveis (pelo menos uma).	5 grupos / 14 opções	Perfis, TiposDeInteração, Fechamento, FaixasDeAviso, FormatoDeExportação
Alternative Group	Obrigar exatamente uma opção entre excludentes.	3 grupos / 7 opções	ModeloDePermissões, BancoDeDados, ServiçoDeEmail

Abstract (abstratas)

São características que só organizam o modelo. Não correspondem a uma funcionalidade isolada: o que existe de fato está nas filhas.

- **Módulos (8):** Autenticação, ControleDeAcesso, Clientes, Funil, Tarefas, Notificações, Indicadores e Infraestrutura. Cada um corresponde a um bloco do sistema (SPEC-01 a SPEC-04 e a base técnica da SPEC-00).
- **Pontos de variação:** Perfis, ModeloDePermissões, TiposDeInteração, Fechamento, FaixasDeAviso, FormatoDeExportação, BancoDeDados e ServiçoDeEmail. Cada uma

representa uma pergunta (“quais perfis? onde ficam as permissões? qual banco?”) e as filhas são as respostas.

- **Agrupadores:** Interações e Relatórios agrupam recursos relacionados dentro de um módulo.

Lista completa (19): MiniCRM, Autenticação, ControleDeAcesso, Perfis, ModeloDePermissões, Clientes, Interações, TiposDeInteração, Funil, Fechamento, Tarefas, Notificações, FaixasDeAviso, Indicadores, Relatórios, FormatoDeExportação, Infraestrutura, BancoDeDados, ServiçoDeEmail.

Concrete (concretas)

São as 55 características que realmente mudam o produto entregue: um recurso (BuscaEFiltros, KanbanDoFunil), uma variante (MySQL, PostgreSQL, H2) ou uma regra (BloqueioPorTentativas). Cada uma tem uma origem rastreável em um RF/RNF ou na spec (coluna “Fonte” da seção 1.3).

Mandatory (obrigatórias)

Estão em todo produto quando o pai está presente (26 características, fora a raiz). As concretas obrigatórias são o núcleo do CRM:

- **LoginEmailSenha (RF01) e SeedDoAdministrador (RF32):** sem login e sem o primeiro administrador ninguém entra no sistema.
- **CadastroDeClientes e BuscaEFiltros (RF03, RF04):** o cliente é a entidade central do CRM, e uma carteira sem busca é inutilizável.
- **EscopoDeCarteira (RF15, RF26):** cada usuário precisa enxergar só o que lhe pertence; é o ativo que os 4 módulos reutilizam.
- **CadastroDeOportunidades e KanbanDoFunil (RF07 a RF09):** o funil é a razão de existir do sistema.
- **CadastroDeTarefas e ConclusãoDeTarefa (RF10, RF28), Dashboard (RF11) e PaginaçãoDeListagens (RF18):** completam o fluxo diário do vendedor.
- **ApiRest:** o backend é uma API REST em /api/v1, base de qualquer interface.

Optional (opcionais)

Podem ou não estar presentes (26 características). O produto continua válido sem elas, por isso são diferenciais:

- **Segurança extra:** BloqueioPorTentativas, RecuperaçãoDeSenha, TrocaDeSenhaNoPrimeiroAcesso. Um produto interno simples poderia dispensá-las.
- **Gestão:** GestãoDeUsuários, VínculoGerenteVendedor, TransferênciaDeVendedor e ExclusãoLógica/ExclusãoFísica, que dependem do modelo de equipe adotado.
- **Experiência e controle:** MotivoDaPerda, ArrastarESoltar, HistóricoDeEtapas, ReaberturaDeOportunidade, TotaisPorEtapa, HistóricoDoCliente, StatusAutomático.
- **Módulos opcionais inteiros:** Notificações, Indicadores e Relatórios são opcionais porque um CRM mínimo funciona sem eles.
- **Infraestrutura:** InterfaceWeb (o produto pode ser só API) e BackupDoBanco.

Or Group (grupos OR)

Em um grupo Or, pelo menos uma opção deve ser escolhida e várias podem coexistir. Foi usado onde as opções não se excluem:

Grupo	Opções	Justificativa
Perfis	Administrador, Gerente, Vendedor	O produto precisa de ao menos um perfil, mas pode ter vários ao mesmo tempo, como o sistema atual (RF02).

Grupo	Opções	Justificativa
TiposDeInteração	Ligação, EmailEnviado, Reunião	Um cliente pode ser contatado por qualquer canal; o produto oferece um ou mais (RF05).
Fechamento	Ganha, Perdida	A oportunidade pode fechar de duas formas, e o produto pode suportar uma ou as duas (RF08).
FaixasDeAviso	Aviso15Dias, AvisoVéspera, AvisoNoDia, AvisoVencida	As faixas de vencimento são independentes e combináveis (RF13).
FormatoDeExportação	ExportaçãoCSV, ExportaçãoPDF	O relatório pode sair em CSV, em PDF ou nos dois (RF12: CSV obrigatório, PDF desejável).

Alternative Group (grupos alternativos)

Em um grupo Alternative, exatamente uma opção é escolhida. Foi usado onde a escolha é excludente por natureza:

Grupo	Opções	Justificativa
ModeloDePermissões	PermissõesNoCódigo, PermissõesComoDados	As permissões ficam no código (enum Permissoes, solução atual) ou em tabelas do banco (proposta da SPEC-01); não faz sentido ter as duas fontes de verdade.
BancoDeDados	MySQL, PostgreSQL, H2	Um produto roda sobre um único SGBD. O MySQL é o atual (RNF06); H2 serve a desenvolvimento e testes.
ServiçoDeEmail	EmailEmLog, EmailSmtp	Há uma única implementação de EmailService ativa por produto: o adapter que só registra em log (atual) ou o envio SMTP. O grupo inteiro é opcional.

1.5 Restrições (constraints)

Foram elaboradas 14 restrições, escritas no editor de constraints do FeatureIDE com os operadores Implies (⇒), And (∧), Or (∨) e Not (¬). Elas expressam o que a árvore sozinha não consegue dizer: dependências entre módulos, perfis e interface, e incompatibilidades.

ID	Fórmula	Operadores	Tipo	Justificativa (fonte)
C1	RecuperaçãoDeSenha ⇒ ServiçoDeEmail	Implies	Dependência (requires)	A recuperação de senha envia um código por e-mail; sem um serviço de e-mail (hoje o adapter EmailEmLog) o recurso não funciona. (RF17)
C2	MotivoDaPerda ⇒ Perdida	Implies	Dependência (requires)	O motivo da perda só faz sentido quando a oportunidade é fechada como perdida (preço, concorrente, sem verba, sem resposta). (RF29)
C3	TransferênciaDeVendedor ⇒ Gerente ∨ Administrador	Implies, Or	Dependência por perfil (Or)	Transferir cliente é ação de quem gerencia carteiras; o produto precisa ter Gerente ou Administrador (permissão CLIENTE_TRANSFERIR). (RF26)
C4	VínculoGerenteVendedor ⇒ Gerente ∧ Vendedor	Implies, And	Dependência composta (And)	O vínculo gerente→vendedor exige os dois perfis no produto: o gerente que cria e o vendedor que fica vinculado (coluna id_gerente). (RF27)
C5	RankingDeVendedores ⇒ Dashboard ∧ (Gerente ∨ Administrador)	Implies, And, Or	Dependência composta (And + Or)	O ranking mostra o resultado do dashboard e só é acessível a Gerente (sua equipe) e Administrador (geral); o vendedor não o acessa. (RF31)

ID	Fórmula	Operadores	Tipo	Justificativa (fonte)
C6	Relatórios ⇒ Gerente ∨ Administrador	Implies, Or	Dependência por perfil (Or)	Relatórios e exportações são restritos a Gerente e Administrador; o vendedor não exporta. (RF22)
C7	ArrastarESoltar ⇒ InterfaceWeb ∧ KanbanDoFunil	Implies, And	Dependência composta (And)	Arrastar e soltar é um comportamento de interface que depende do frontend web e do quadro Kanban do funil. (RNF09)
C8	PopUpDeVencimento ⇒ InterfaceWeb	Implies	Dependência (requires)	O pop-up de vencimento aparece na tela; sem interface web não há onde exibi-lo. (RF24)
C9	GestãoDeUsuários ⇒ Administrador ∨ Gerente	Implies, Or	Dependência por perfil (Or)	Cadastrar e desativar usuários é função de Administrador ou de Gerente (que cria vendedores). (RF14)
C10	PermissõesComoDados ⇒ GestãoDeUsuários	Implies	Dependência (requires)	Permissões guardadas como dados só são úteis se existir a tela de gestão/liberação de acessos para alterá-las. (RF25)
C11	¬(ExclusãoLógica ∧ ExclusãoFísica)	Not, And	Exclusão mútua (Not + And)	A exclusão de cliente é lógica (preserva histórico) ou física, mas nunca as duas ao mesmo tempo no mesmo produto. (RF03)
C12	BackupDoBanco ⇒ ¬H2	Implies, Not	Incompatibilidade (Not)	O backup periódico exige banco persistente; o H2 em memória perde os dados ao reiniciar, então não pode ser combinado. (RNF06)
C13	StatusAutomático ⇒ Ganha	Implies	Dependência (requires)	O status automático (cliente vira ATIVO) é disparado pela primeira oportunidade ganha, então o desfecho Ganha precisa existir. (RF30)
C14	AvisoDeReunião ⇒ Reunião	Implies	Dependência (requires)	O aviso de reunião nasce de interações do tipo reunião; sem esse tipo de interação não há o que avisar. (RF23)

Cobertura dos operadores exigidos:

Operador	Constraints	Exemplo
Implies (⇒)	C1 a C14 (todas)	C1: RecuperaçãoDeSenha ⇒ ServiçoDeEmail
And (∧)	C4, C5, C7, C11	C4: VínculoGerenteVendedor ⇒ Gerente ∧ Vendedor
Or (∨)	C3, C5, C6, C9	C6: Relatórios ⇒ Gerente ∨ Administrador
Not (¬)	C11, C12	C12: BackupDoBanco ⇒ ¬H2

1.6 Validação no FeatureIDE: o produto atual é uma configuração válida

Para provar que o modelo descreve o sistema real, o produto atual foi registrado como configuração (configs/MiniCRM-atual.xml): 47 características concretas selecionadas e 8 não selecionadas. O editor de configuração do FeatureIDE confirmou: valid, 1 possible configurations.

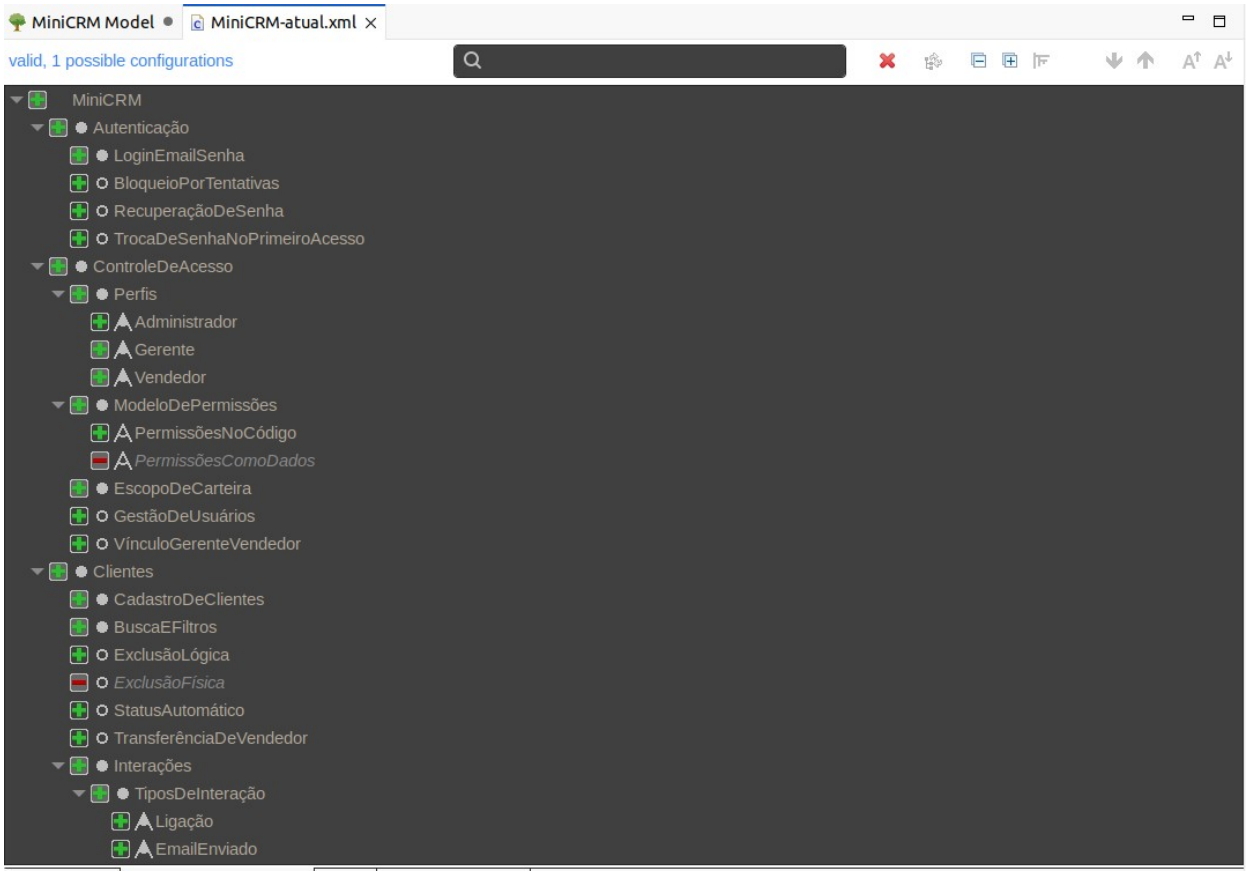


Figura 5 – Editor de configuração do FeatureIDE: “valid, 1 possible configurations”.

Características do modelo que o produto atual ainda não implementa (variabilidade em aberto): PermissõesComoDados, ExclusãoFísica, PopUpDeVencimento, ExportaçãoPDF, PostgreSQL, H2, EmailSmtp, BackupDoBanco.

Durante a modelagem, uma primeira versão da configuração (arquivo escrito sem os atributos de seleção) foi marcada pelo FeatureIDE como inválida; após corrigir o formato, a validação passou. O painel Problems termina sem erros, apenas com 4 avisos informativos.

Parte II – MPS.BR: GRU e DRU

A classificação segue o Guia Geral MPS de Software:2016. A Gerência de Reutilização (GRU, página 36, nível E) e o Desenvolvimento para Reutilização (DRU, página 42, nível C) foram avaliados contra o que existe de fato no repositório MiniCRM-Reuso (branch main, antes desta entrega).

Cor	Significado
Verde	De acordo com o resultado esperado.
Amarelo	Identificação parcial: parte da prática existe.
Vermelho	Não encontrado no projeto.

2.1 GRU – Gerência de Reutilização

Propósito (p. 36): gerenciar o ciclo de vida dos ativos reutilizáveis. Resultado do projeto: 0 verde · 4 amarelo · 1 vermelho.

Resultado	O que o guia espera	Cor	Justificativa e evidência
GRU 1	Estratégia de gerenciamento de ativos documentada (definição de ativo, critérios de aceitação, certificação, classificação, descontinuidade e avaliação).	**Amarelo**	Há identificação parcial de ativos: o README.md lista os padrões de projeto reutilizados (Template Method em SeedBase, AbstractExportadorService e AbstractNotificacaoService; Strategy, Observer, State, Specification e Facade) e o docs/specs/README.md traz o “Mapa de integração”, com as interfaces de serviço públicas e seus donos (EscopoCarteira, ClienteConsultaService, OportunidadeConsultaService). Falta o documento de estratégia: não existe definição formal de ativo reutilizável nem critérios de aceitação, certificação, classificação, descontinuidade e avaliação.
GRU 2	Mecanismo de armazenamento e recuperação de ativos reutilizáveis implantado.	**Amarelo**	O armazenamento existe: repositório Git com 52 commits, 11 pull requests mesclados e uma branch por spec. A recuperação é manual (README e docs/specs). Não há catálogo de ativos nem repositório de artefatos: tudo vive em um único módulo Maven, na versão 0.0.1-SNAPSHOT, sem publicação em Nexus ou GitHub Packages.
GRU 3	Dados de utilização dos ativos reutilizáveis registrados.	**Vermelho**	Nenhum dado de utilização é registrado: não há contagem de consumidores, log de uso ou métrica de reutilização. A tabela “Interface × Consumidores” do docs/specs/README.md é um desenho feito antes do código, não um registro de uso real. (Medição pontual feita nesta atividade: PageResponse e EscopoCarteira aparecem em 12 arquivos cada, incluindo a própria definição.)
GRU 4	Ativos mantidos periodicamente conforme critérios definidos, com modificações controladas ao longo do ciclo de vida.	**Amarelo**	As modificações são controladas: commits no padrão Conventional Commits, pull request para a main com revisão pelo dev da spec vizinha e a regra “ninguém altera um payload sem PR aprovado pelos 4” (docs/specs/README.md). Falta a manutenção periódica por critérios: o repositório não tem tags nem releases (git tag vazio), não tem CHANGELOG, a versão é fixa em 0.0.1-SNAPSHOT e não há política de revisão dos ativos.

Resultado	O que o guia espera	Cor	Justificativa e evidência
GRU 5	Usuários dos ativos notificados sobre problemas, modificações, novas versões e descontinuidade.	**Amarelo**	A notificação de modificações acontece de modo informal, pelo fluxo de pull requests do GitHub (o revisor vizinho é avisado de cada mudança). Não há comunicação de problemas conhecidos, de novas versões nem de descontinuidade: sem release notes, sem aviso de depreciação e sem canal definido para os consumidores dos ativos.

Como adequar o projeto à GRU (GRU 1 a 5)

Resultado	Ação proposta
GRU 1	Criar docs/reuso/ESTRATEGIA-DE-REUSO.md com: definição de ativo (biblioteca, template, spec, feature model); aceitação (testes passando e documentação); certificação (PR aprovado por revisor vizinho); classificação por domínio (segurança, API, persistência, relatórios); descontinuidade (aviso de uma sprint e @Deprecated); avaliação (taxa de reuso por sprint).
GRU 2	Extrair os ativos para um módulo minicrm-commons (PageResponse, ApiExceptionHandler, EscopoCarteira, SeedBase, ClienteSpecs) publicado em um repositório Maven (GitHub Packages ou Nexus), e criar docs/reuso/CATALOGO-DE-ATIVOS.md com nome, versão, dono, caminho, consumidores e exemplo de uso.
GRU 3	Registrar o uso: coluna “Consumidores” do catálogo atualizada por checklist no template de PR, mais um script no CI que conta os imports de cada ativo e grava o resultado em um arquivo versionado.
GRU 4	Adotar versionamento semântico (tags v0.1.0, v0.2.0…), CHANGELOG.md gerado a partir dos Conventional Commits, arquivo CODEOWNERS por ativo e revisão dos ativos a cada sprint, com os critérios da estratégia.
GRU 5	Publicar GitHub Releases com notas de versão, marcar ativos obsoletos com @Deprecated e aviso no PR, e definir um canal (Teams/WhatsApp do grupo) para avisos de problemas, novas versões e descontinuidade; o CODEOWNERS pede revisão automática dos consumidores.

2.2 DRU – Desenvolvimento para Reutilização

Propósito (p. 42): identificar oportunidades de reutilização sistemática e, se possível, estabelecer um programa de reutilização com engenharia de domínio. Os nove resultados foram classificados; os resultados 6 a 9 (modelagem e arquitetura de domínio) são os que esta entrega altera, então têm duas colunas: antes e depois.

Resultado	O que o guia espera	Antes	Depois	Justificativa e evidência
DRU 1	Domínios de aplicação com potencial de reutilização identificados.	**Amarelo**	**Amarelo**	O domínio (CRM para equipes de vendas) está identificado no README, e o docs/specs/README.md cita o PDF “Project Planning for Software Product Lines”, com ativos e pontos de variação (PV1–PV12). Esse PDF fica fora do repositório e a parte de alta variabilidade foi adiada (“fica fora desta fase”).
DRU 2	Capacidade de reutilização sistemática avaliada, com ações corretivas.	**Vermelho**	**Vermelho**	Não há avaliação da capacidade de reutilização (maturidade, lacunas, ações corretivas) em nenhum documento do projeto.
DRU 3	Programa de reutilização planejado (propósitos, escopo, metas, objetivos).	**Vermelho**	**Vermelho**	Não existe programa de reutilização. O “Escopo desta fase” do docs/specs/README.md é o escopo do projeto (somente backend), não de um programa de reuso.
DRU 4	Programa de reutilização implantado, monitorado e avaliado.	**Vermelho**	**Vermelho**	Sem programa (DRU 3), não há implantação, monitoramento nem avaliação.

Resultado	O que o guia espera	Antes	Depois	Justificativa e evidência
DRU 5	Propostas de reutilização avaliadas quanto à adequação à aplicação alvo.	**Amarelo**	**Amarelo**	A reutilização do Ativo 6 (RBAC) foi avaliada e adaptada: a SPEC-01 propôs permissões como dados (tabelas permissao e perfil_permissao) e a implementação ficou com o enum Permissoes e @PreAuthorize por autoridade (README: “controle de acesso por permissão, não por cargo”). A decisão não está registrada como avaliação formal, e o banco real tem 9 tabelas, sem tabela de permissões.
DRU 6	Formas de representação de modelos e arquiteturas de domínio selecionadas.	**Amarelo**	**Verde**	Antes: existiam o diagrama ER em Mermaid (README) e a tabela de padrões de projeto, mas nenhuma notação para variabilidade. Depois (esta entrega): foram escolhidos o feature model do FeatureIDE para a variabilidade e o diagrama ER em Mermaid para os dados, com a escolha registrada neste documento.
DRU 7	Modelo de domínio desenvolvido, com limites e relações definidos, capaz de capturar características comuns, variantes, opcionais e obrigatórias.	**Vermelho**	**Verde**	Antes: não havia modelo de variabilidade no repositório. Depois (esta entrega): model.xml com 74 características (obrigatórias, opcionais, grupos Or e Alternative) e 14 constraints; a configuração do produto atual foi validada pelo FeatureIDE (“valid, 1 possible configurations”). Os limites do domínio vêm do Documento de Requisitos, seção 2.3 (fora do escopo: e-mail marketing e integrações externas, exceto o e-mail transacional), e o modelo fica versionado no Git.
DRU 8	Arquitetura de domínio descrevendo uma família de aplicações, mantida ao longo do ciclo de vida.	**Amarelo**	**Amarelo**	A arquitetura modular está documentada: pacotes controller, service, repository, model e outros; eventos de domínio e interfaces de consulta entre módulos (SPEC-00). Ela descreve o produto, não uma família: não liga os pontos de variação a componentes (EmailService, AbstractExportadorService, ModeloDePermissões) nem traz guia de derivação de produtos.
DRU 9	Ativos do domínio especificados, desenvolvidos e mantidos ao longo do ciclo de vida.	**Amarelo**	**Amarelo**	Existem ativos de domínio especificados (SPEC-00: paginação, erros, eventos, seeds) e implementados (PageResponse, EscopoCarteira, SeedBase, AbstractNotificacaoService, ClienteSpecs). Porém ficam em um único módulo, sem versão própria, e algumas variantes previstas ainda não existem (exportação PDF, e-mail SMTP, permissões como dados).

Resumo antes → depois: 0 verde · 5 amarelo · 4 vermelho → 2 verde · 4 amarelo · 3 vermelho. A mudança vem do feature model e da configuração validada entregues na Parte I (DRU 6 e DRU 7); as demais práticas dependem de ações de gestão, listadas a seguir.

Como adequar o projeto à DRU (DRU 1 a 9 que ainda não estão verdes)

Resultado	Ação proposta
DRU 1	Trazer para docs/reuso/ um resumo do PDF de planejamento da linha de produtos (ativos e pontos de variação PV1–PV12) e listar os domínios candidatos a reuso (segurança/RBAC, paginação, notificações, relatórios).
DRU 2	Fazer uma autoavaliação de maturidade de reuso por sprint (checklist com os resultados GRU/DRU deste documento) e registrar ações corretivas.
DRU 3	Escrever um plano de programa de reuso de uma página: propósito, escopo (4 módulos), metas (por exemplo, 3 ativos extraídos em módulo Maven) e responsáveis.
DRU 4	Executar o plano, acompanhar as metas a cada sprint e registrar o resultado em docs/reuso/.

Resultado	Ação proposta
DRU 6	Registrar em docs/reuso/ as representações adotadas: feature model do FeatureIDE (variabilidade), ER em Mermaid (dados) e diagrama de componentes (arquitetura). Esta entrega já cumpre a primeira parte.
DRU 7	Manter docs/reuso/featureide/model.xml no repositório e exigir, no checklist de PR, a atualização do modelo quando uma funcionalidade entrar ou sair; validar a configuração do produto atual a cada release.
DRU 8	Criar docs/reuso/ARQUITETURA-DE-DOMINIO.md com o mapa característica → pacote/classe (por exemplo, ExportaçãoCSV → CsvExportadorService, Notificações → AbstractNotificacaoService) e os pontos de extensão (EmailService, AbstractExportadorService, ModeloDePermissões), com um guia curto de derivação de produto por configuração.
DRU 9	Implementar as variantes previstas no modelo (ExportaçãoPDF como nova subclasse de AbstractExportadorService, EmailSmtp como nova implementação de EmailService e PermissõesComoDados) e empacotar os ativos comuns em minicrm-commons, com testes por ativo e versionamento próprio.

Parte III – Database First (.NET)

3.1 O que foi feito

O backend principal do Mini CRM é code first: as entidades Java existem primeiro e o Hibernate cria as tabelas (ddl-auto=update). Nesta etapa o fluxo foi invertido: partimos do banco mini_crm_db, que já existe com suas 9 tabelas, e geramos o código a partir dele em .NET, até o CRUD e as telas.

Peça	Ferramenta	O que gera
Banco de dados	MySQL 8.4.13 em Docker (docker-compose.yml)	As 9 tabelas do Mini CRM com dados de exemplo
Entidades e DbContext	EF Core Scaffolding (dotnet ef dbcontext scaffold, provider Pomelo 9.0.0)	9 classes em Models/ e o MiniCrmContext
Telas	ASP.NET Core aspnet-codegenerator (controller MVC)	9 controllers MVC e 45 views Razor (Index, Details, Create, Edit, Delete)
API REST	aspnet-codegenerator (controller -api) e Swagger	9 controllers de API (GET, POST, PUT, DELETE) documentados em /swagger

O EF Core Scaffolding é a ferramenta oficial da Microsoft para Database First, e o aspnet-codegenerator gera telas Razor de verdade (formulários de listar, criar, editar e excluir), não só uma API. Todo o código de Models/, Data/, Controllers/ e Views/ foi produzido pelas ferramentas. O que foi escrito à mão: Program.cs, Design/DesignTimeServices.cs (pluralizador em português), appsettings.json, o .csproj e os scripts em dbfirst/scripts/.

Um ponto de arquitetura que vale destacar: os controllers gerados falam direto com o DbContext. Não existe camada de service. No backend real, o controller chama um service, que aplica as regras (escopo de carteira, transições do funil) e só então chega ao repositório.

3.2 Ambiente e versões

Componente	Versão e origem
SDK do .NET	9.0.321, instalado em ~/.dotnet pelo script oficial dotnet-install (sem sudo)
dotnet-ef	9.0.23 (dotnet tool, em ~/.dotnet-tools)
dotnet-aspnet-codegenerator	9.0.13 (dotnet tool)
Provider MySQL	Pomelo.EntityFrameworkCore.MySql 9.0.0 (exige EF Core 9.0.x, o que determinou o SDK 9)
Swagger	Swashbuckle.AspNetCore 10.3.1
Banco	MySQL 8.4.13 (imagem mysql:8.4), porta 3307

3.3 Passo a passo com a saída real dos comandos

Passo 1 – Banco de dados

O esquema e os dados vêm de um mysqldump do banco criado pelo backend real e populado pela API (4 usuários, 6 clientes, 6 interações, 6 oportunidades e 4 tarefas fictícios). O Compose os carrega na primeira subida.

```
$ docker compose down -v && docker compose up -d --wait
 Network dbfirst_default Removing
 Network dbfirst_default Removed
 Container minicrm-mysql-dbfirst Starting
 Container minicrm-mysql-dbfirst Started
 Container minicrm-mysql-dbfirst Waiting
 Container minicrm-mysql-dbfirst Healthy
```

Passo 2 – Entidades e DbContext (EF Core Scaffolding)

O script gerar.sh lê o esquema do MySQL e gera as classes. O comando central é este:

```
dotnet ef dbcontext scaffold "$CONEXAO" Pomelo.EntityFrameworkCore.MySql \
  --project MiniCrm.DbFirst --output-dir Models --context MiniCrmContext --context-dir Data \
  --data-annotations --no-onconfiguring --force
$ ./scripts/gerar.sh
==> EF Core Scaffolding: lendo o esquema de mini_crm_db em localhost:3307 e gerando as classes
Using ServerVersion '8.4.13-mysql'.

Entidades geradas em MiniCrm.DbFirst/Models:
  Cliente.cs
  HistoricoEtapa.cs
  Interacao.cs
  MotivoPerda.cs
  Notificacao.cs
  Oportunidade.cs
  Tarefa.cs
  TokenRecuperacao.cs
  Usuario.cs
DbContext gerado em MiniCrm.DbFirst/Data/MiniCrmContext.cs
```

Passo 3 – CRUD e telas (aspnet-codegenerator)

O script gerar-crud.sh percorre as 9 entidades e, para cada uma, roda o gerador duas vezes. Para a entidade Cliente, os comandos são:

```
dotnet-aspnet-codegenerator controller -name ClientesController -m Cliente -dc MiniCrmContext \
  --relativeFolderPath Controllers --useDefaultLayout --referenceScriptLibraries -f      # telas
dotnet-aspnet-codegenerator controller -name ClientesController -m Cliente -dc MiniCrmContext \
  -api --relativeFolderPath Controllers/Api -f                                           # API
$ ./scripts/gerar-crud.sh
==> Gerando CRUD (controller MVC + views + controller de API) para 9 entidades
  Cliente           -> tela /Clientes (5 views)  API /api/Clientes
  HistoricoEtapa    -> tela /HistoricoEtapas (5 views)  API /api/HistoricoEtapas
  Interacao         -> tela /Interacoes (5 views)  API /api/Interacoes
  MotivoPerda       -> tela /MotivoPerdas (5 views)  API /api/MotivoPerdas
  Notificacao       -> tela /Notificacoes (5 views)  API /api/Notificacoes
  Oportunidade      -> tela /Oportunidades (5 views)  API /api/Oportunidades
  Tarefa            -> tela /Tarefas (5 views)   API /api/Tarefas
  TokenRecuperacao  -> tela /TokenRecuperacoes (5 views)  API /api/TokenRecuperacoes
  Usuario           -> tela /Usuarios (5 views)  API /api/Usuarios
Pronto. Compile e rode com:  ./scripts/rodar.sh
```

Passo 4 – Aplicação no ar

```
$ ./scripts/rodar.sh
Now listening on: http://localhost:8095
Application started. Press Ctrl+C to shut down.

$ curl http://localhost:8095/swagger/v1/swagger.json   (recursos da API)
   /api/Clientes
   /api/HistoricoEtapas
   /api/Interacoes
   /api/MotivoPerdas
   /api/Notificacoes
   /api/Oportunidades
   /api/Tarefas
   /api/TokenRecuperacoes
   /api/Usuarios
```

Passo 5 – CRUD pela API contra o MySQL

O script demo-crud.sh lê os clientes, cria um, altera o status e o exclui, tudo pela API gerada. Ao final o banco volta a ter os 6 clientes de exemplo.

```
$ ./scripts/demo-crud.sh
1) READ   - listando clientes que já estão no banco
   - Padaria Pão Quente | contato@paoquente.com.br | PROSPECT
   - Oficina Mecânica Rápida | oficina@rapida.com.br | PROSPECT
   - Clínica Sorriso | clinica@sorriso.com.br | ATIVO
   - Escola Aprender Mais | secretaria@aprendermais.com.br | PROSPECT
   - Mercado Bom Preço | compras@bompreco.com.br | PROSPECT
   - Studio Fit | recepcao@studiofit.com.br | PROSPECT

2) CREATE - criando um cliente pela API gerada
   criado com id 7

3) UPDATE - mudando o status de PROSPECT para ATIVO (PUT com o objeto)
   PUT respondeu HTTP 204
   agora: Cliente Criado pela API .NET | ATIVO

4) DELETE - removendo o cliente
   DELETE respondeu HTTP 204
   GET depois de excluir: HTTP 404

CRUD completo executado nas tabelas do MySQL.
```

Passo 6 – CRUD pelas telas Razor

O script demo-telas.sh faz o mesmo pelos formulários gerados, como o navegador: abre a página, lê o token antiforgery e envia o POST. A resposta 302 é o redirecionamento de volta à lista, que o ASP.NET devolve quando o formulário foi aceito.

```
$ ./scripts/demo-telas.sh
Clientes no banco antes: 6

1) CREATE pela tela  (GET /Clientes/Create -> POST do formulário)
   POST /Clientes/Create: 302 -> http://localhost:8095/
   cliente criado com id 8; total agora: 7

2) UPDATE pela tela  (GET /Clientes/Edit/8 -> POST do formulário)
   POST /Clientes/Edit/8: 302 -> http://localhost:8095/
   agora: Cliente Criado pela Tela Razor | ATIVO

3) DELETE pela tela  (GET /Clientes/Delete/8 -> POST de confirmação)
   POST /Clientes/Delete/8: 302 -> http://localhost:8095/
   GET depois de excluir: HTTP 404; total agora: 6

CRUD pelas telas executado nas tabelas do MySQL.
```

3.4 Código gerado

Foram gerados 9 arquivos de entidade e o DbContext (612 linhas), 9 controllers MVC (1511 linhas), 45 views Razor e 9 controllers de API (972 linhas), nenhum escrito à mão.

Entidade	Tabela	Colunas	Linhas	Tela (MVC)	API
Cliente	cliente	10	63	/Clientes	/api/Clientes
HistoricoEtapa	historico_etapa	6	41	/HistoricoEtapas	/api/HistoricoEtapas
Interacao	interacao	7	44	/Interacoes	/api/Interacoes
MotivoPerda	motivo_perda	2	23	/MotivoPerdas	/api/MotivoPerdas
Notificacao	notificacao	7	46	/Notificacoes	/api/Notificacoes
Oportunidade	oportunidade	11	70	/Oportunidades	/api/Oportunidades
Tarefa	tarefa	10	62	/Tarefas	/api/Tarefas
TokenRecuperacao	token_recuperacao	6	37	/TokenRecuperacoes	/api/ TokenRecuperacoes
Usuario	usuario	13	88	/Usuarios	/api/Usuarios

Entidade Cliente (gerada)

As anotações de tabela e coluna, o enum do status, a chave estrangeira para o vendedor e as coleções de interações, oportunidades e tarefas saíram do esquema do banco.

```
[Table("cliente")]
[Index("IdVendedor", Name = "FKsn714u8ael2eiyov98ub9vvp1")]
public partial class Cliente
{
    [Key]
    [Column("id_cliente")]
    public int IdCliente { get; set; }

    [Column("atualizado_em")]
    public DateTime AtualizadoEm { get; set; }

    [Column("criado_em")]
    public DateTime CriadoEm { get; set; }

    [Required]
    [Column("email")]
    [StringLength(160)]
    public string Email { get; set; }
```

```
    [Column("empresa")]
    [StringLength(120)]
    public string Empresa { get; set; }

    [Column("excluido", TypeName = "bit(1)")]
    public ulong Excluido { get; set; }

    [Required]
    [Column("nome")]
    [StringLength(120)]
    public string Nome { get; set; }

    [Required]
    [Column("status", TypeName = "enum('ATIVO','INATIVO','PROSPECT')")]
    public string Status { get; set; }

    [Column("telefone")]
    [StringLength(20)]
    public string Telefone { get; set; }

    [Column("id_vendedor")]
    public int IdVendedor { get; set; }

    [ForeignKey("IdVendedor")]
    [InverseProperty("Clientes")]
    public virtual Usuario IdVendedorNavigation { get; set; }

    [InverseProperty("IdClienteNavigation")]
    public virtual ICollection<Interacao> Interacoes { get; set; } = new List<Interacao>();

    [InverseProperty("IdClienteNavigation")]
    public virtual ICollection<Oportunidade> Oportunidades { get; set; } = new List<Oportunidade>();

    [InverseProperty("IdClienteNavigation")]
    public virtual ICollection<Tarefa> Tarefas { get; set; } = new List<Tarefa>();
}
```

DbContext: configuração de Cliente (gerada)

```
modelBuilder.Entity<Cliente>(entity =>
        {
            entity.HasKey(e => e.IdCliente).HasName("PRIMARY");

            entity.HasOne(d => d.IdVendedorNavigation).WithMany(p => p.Clientes)
                .OnDelete(DeleteBehavior.ClientSetNull)
                .HasConstraintName("FKsn714u8ael2eiyov98ub9vvp1");
        });
```

Controller MVC de Clientes (gerado)

Usa o MiniCrmContext direto; o dropdown de vendedor do formulário é montado a partir da chave estrangeira.

```
public class ClientesController : Controller
    {
        private readonly MiniCrmContext _context;

        public ClientesController(MiniCrmContext context)
        {
            _context = context;
        }
```

```
        // GET: Clientes
        public async Task<IActionResult> Index()
        {
            var miniCrmContext = _context.Clientes.Include(c => c.IdVendedorNavigation);
            return View(await miniCrmContext.ToListAsync());
        }

        // GET: Clientes/Details/5
        public async Task<IActionResult> Details(int? id)
        {
            if (id == null)
            {
                return NotFound();
            }

            var cliente = await _context.Clientes
                .Include(c => c.IdVendedorNavigation)
                .FirstOrDefaultAsync(m => m.IdCliente == id);
            if (cliente == null)
            {
                return NotFound();
            }

            return View(cliente);
        }

        // GET: Clientes/Create
        public IActionResult Create()
        {
            ViewData["IdVendedor"] = new SelectList(_context.Usuarios, "IdUsuario", "Email");
            return View();
        }

        // POST: Clientes/Create
        // To protect from overposting attacks, enable the specific properties you want to bind to.
        // For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult>
Create([Bind("IdCliente,AtualizadoEm,CriadoEm,Email,Empresa,Excluido,Nome,Status,Telefone,IdVendedor")]
Cliente cliente)
        {
            if (ModelState.IsValid)
            {
                _context.Add(cliente);
                await _context.SaveChangesAsync();
                return RedirectToAction(nameof(Index));
            }
            ViewData["IdVendedor"] = new SelectList(_context.Usuarios, "IdUsuario", "Email",
cliente.IdVendedor);
            return View(cliente);
        }
```

View Create.cshtml (gerada)

```
<form asp-action="Create">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="AtualizadoEm" class="control-label"></label>
                <input asp-for="AtualizadoEm" class="form-control" />
                <span asp-validation-for="AtualizadoEm" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="CriadoEm" class="control-label"></label>
                <input asp-for="CriadoEm" class="form-control" />
                <span asp-validation-for="CriadoEm" class="text-danger"></span>
            </div>
            ... (um bloco <div class="form-group"> por coluna) ...
            <div class="form-group">
                <label asp-for="IdVendedor" class="control-label"></label>
                <select asp-for="IdVendedor" class ="form-control" asp-items="ViewBag.IdVendedor"></select>
            </div>
            ...
        </form>
```

Controller de API de Clientes (gerado)

```
[Route("api/[controller]")]
    [ApiController]
    public class ClientesController : ControllerBase
    {
        private readonly MiniCrmContext _context;

        public ClientesController(MiniCrmContext context)
        {
            _context = context;
        }

        // GET: api/Clientes
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Cliente>>> GetClientes()
        {
            return await _context.Clientes.ToListAsync();
        }

        // ... GetCliente(id), PutCliente(id, cliente) ...

// POST: api/Clientes
        // To protect from overposting attacks, see https://go.microsoft.com/fwlink/?linkid=2123754
        [HttpPost]
        public async Task<ActionResult<Cliente>> PostCliente(Cliente cliente)
        {
            _context.Clientes.Add(cliente);
            await _context.SaveChangesAsync();

            return CreatedAtAction("GetCliente", new { id = cliente.IdCliente }, cliente);
        }

        // ... DeleteCliente(id) ...
```

O que foi escrito à mão: Program.cs

Registra o DbContext com o provider MySQL, desliga ciclos no JSON (cliente → oportunidades → cliente) e liga o Swagger. Não há migrations nem EnsureCreated: o banco manda.

```
using System.Text.Json.Serialization;
using Microsoft.EntityFrameworkCore;
using MiniCrm.DbFirst.Data;

var builder = WebApplication.CreateBuilder(args);

// Database First: o banco manda. Não há migrations nem EnsureCreated; o EF só lê e grava dados.
var conexao = builder.Configuration.GetConnectionString("MiniCrm")
    ?? throw new InvalidOperationException("Connection string 'MiniCrm' não encontrada.");
builder.Services.AddDbContext<MiniCrmContext>(options =>
    options.UseMySql(conexao, new MySqlServerVersion(new Version(8, 4, 0))));

builder.Services.AddControllersWithViews()
    // Cliente -> Oportunidades -> Cliente... : sem isso a API entraria em ciclo ao serializar as
associações.
    .AddJsonOptions(o => o.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles);

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
}

// Documentação da API gerada: http://localhost:8095/swagger
app.UseSwagger();
app.UseSwaggerUI();

app.UseRouting();
app.UseAuthorization();

app.MapStaticAssets();

// A tela inicial é a lista de clientes gerada.
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Clientes}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

3.5 Telas geradas (prints reais)

Capturas reais da aplicação gerada em execução, com os dados do MySQL (Chrome em modo headless).

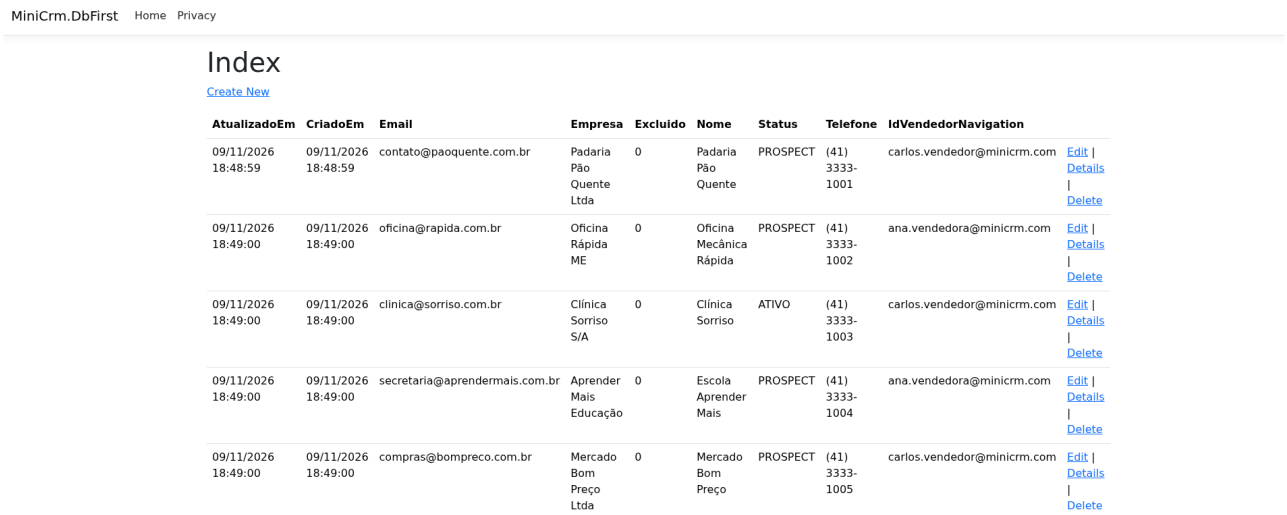


Figura 6 – Lista de clientes gerada (/Clientes): dados do banco e o vendedor resolvido pela chave estrangeira.

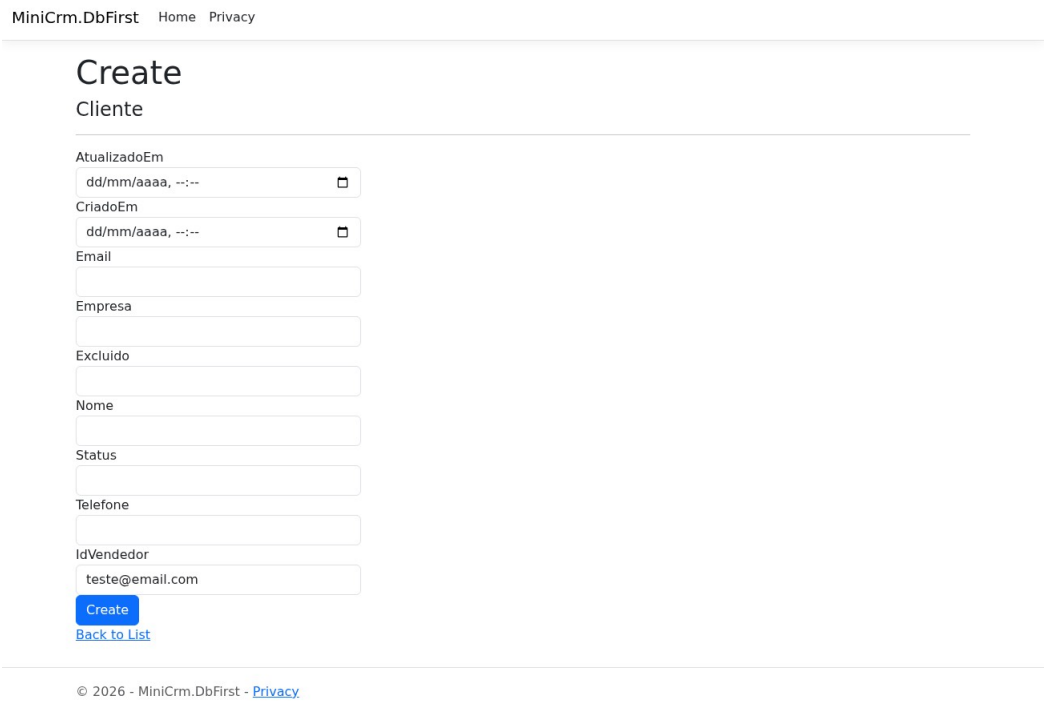


Figura 7 – Formulário de criação gerado (/Clientes/Create), com o dropdown de vendedor montado da chave estrangeira.

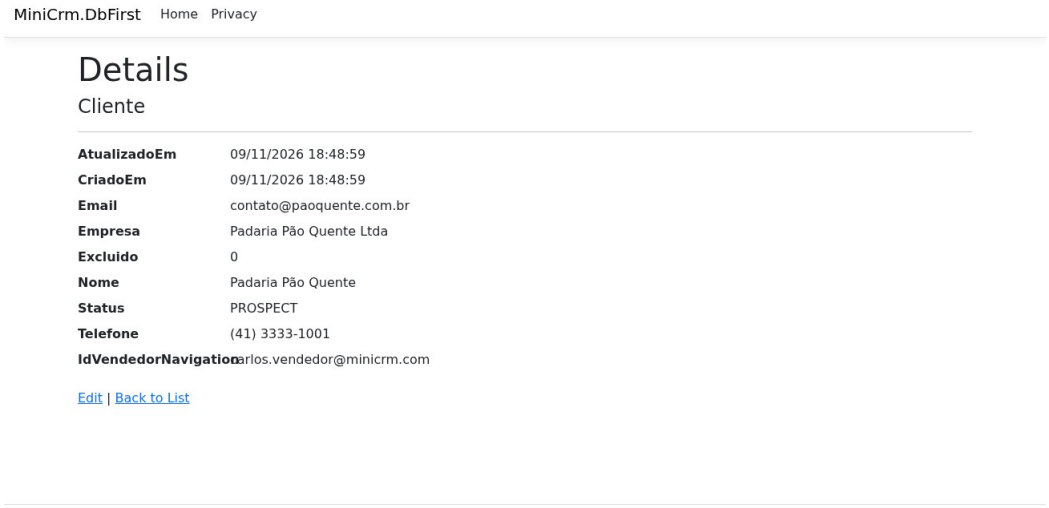


Figura 8 – Tela de detalhes gerada (/Clientes/Details/1).

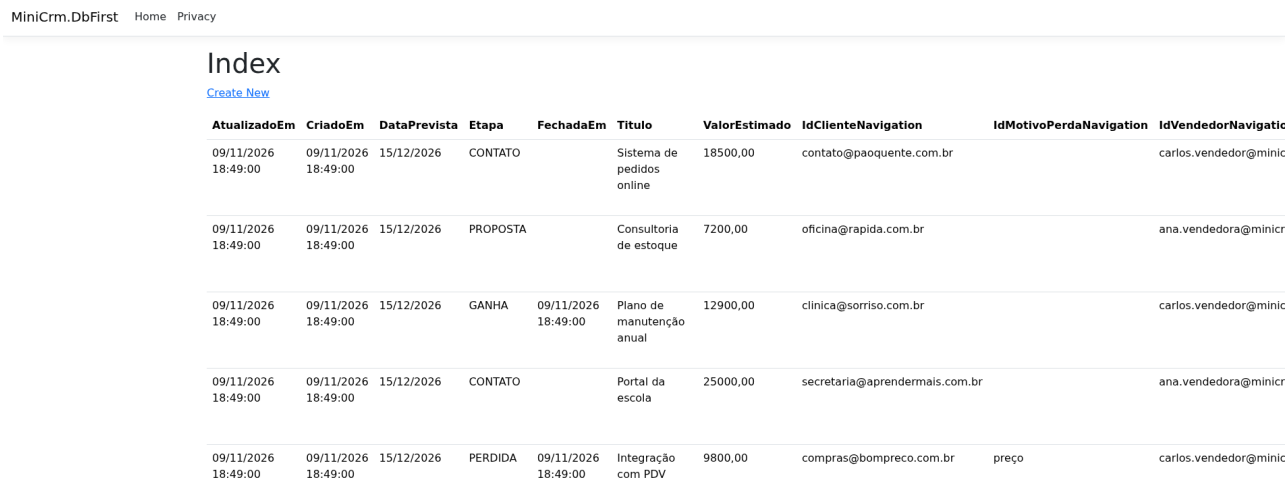


Figura 9 – Lista de oportunidades (o funil de vendas), também gerada (/Oportunidades).

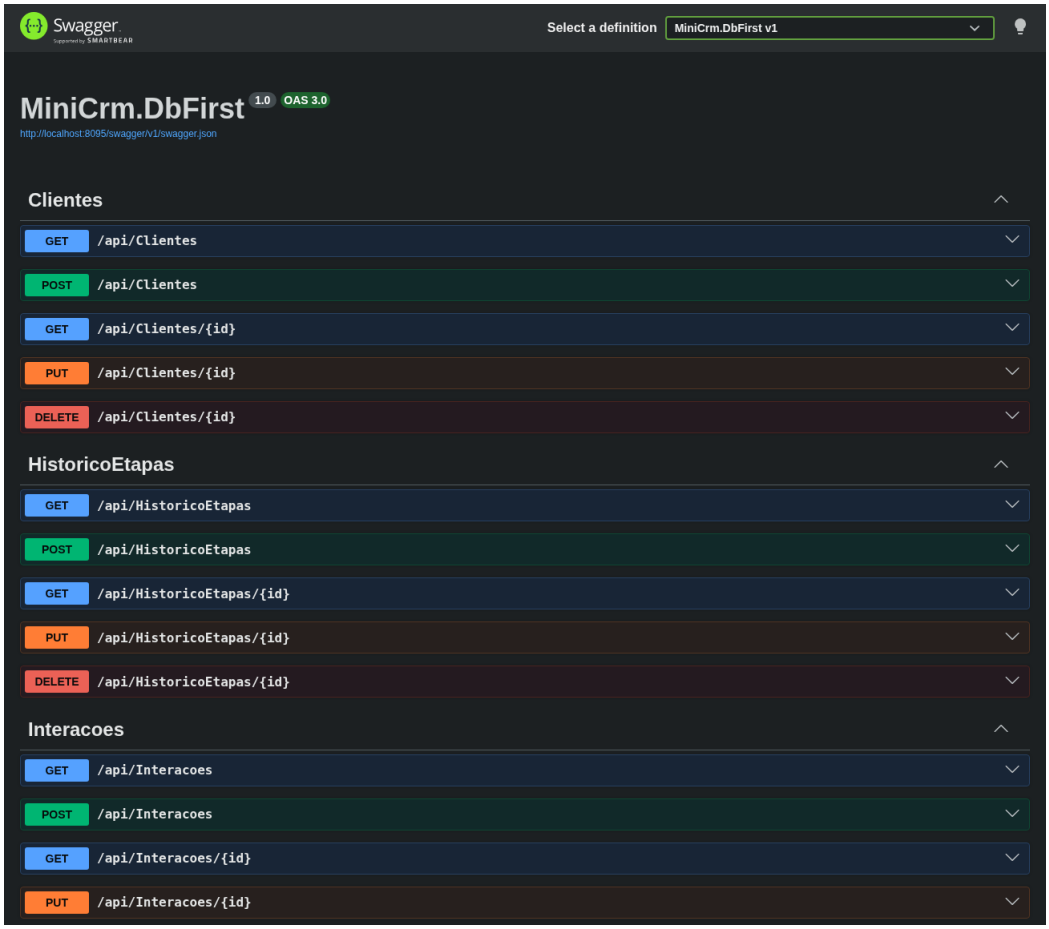


Figura 10 – Swagger (/swagger): os recursos da API gerada, cada um com GET, POST, PUT e DELETE.

3.6 O banco manda: tabela nova vira CRUD sozinha

Para mostrar a inversão do fluxo, uma tabela produto foi criada diretamente no MySQL. Ao rodar os dois scripts de geração, a entidade, os dois controllers e as 5 telas apareceram, sem nenhuma linha escrita. Em seguida a tabela foi removida e o código regenerado.

```
$ mysql> CREATE TABLE produto (...)   -- tabela nova, criada direto no banco
linhas em produto
2

$ ./scripts/gerar.sh && ./scripts/gerar-crud.sh
Entidades geradas em MiniCrm.DbFirst/Models:
  Produto.cs
  Produto            -> Controllers/ProdutosController.cs, Views/Produtos/ (5 telas),
Controllers/Api/ProdutosController.cs  (/api/Produtos)

$ mysql> DROP TABLE produto;   -- volta ao banco original
(tabela removida)

$ ./scripts/gerar.sh && ./scripts/gerar-crud.sh
(regerado)
arquivos de Produto restantes: 0
```

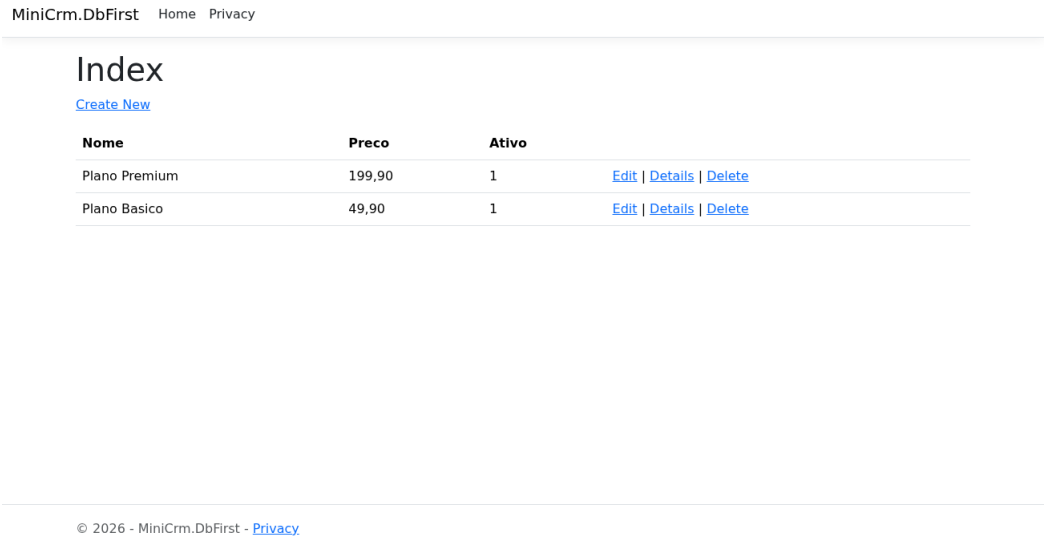


Figura 11 – Tela /Produtos, gerada para a tabela criada direto no banco.

Reprodutibilidade: o ensaio completo (banco novo, geração, aplicação e os dois CRUDs) foi executado do zero duas vezes, e o código gerado foi idêntico nas duas (mesmo hash SHA-256 de todos os arquivos de Models, Data, Controllers e Views: 9830de20…). Depois do teste da tabela nova, regenerar sem ela devolveu o mesmo hash.

3.7 Problemas reais encontrados e como foram resolvidos

Problema	Causa	Solução
O Program.cs não compilava: AddSwaggerGen não existe.	O dotnet add package instalou o Swashbuckle 11.0.0, que só suporta net10.0 e net11.0.	Fixar o Swashbuckle 10.3.1, que cobre net8, net9 e net10.
O aspnet-codegenerator pedia pacotes: primeiro EntityFrameworkCore.Tools e depois EntityFrameworkCore.SqlServer.	O gerador só conhece SQL Server, SQLite, Cosmos e Postgres e exige o pacote do SQL Server mesmo com MySQL. Com ele presente, o log confirma o provider “Pomelo.EntityFrameworkCore.MySql”.	O Tools entra no projeto; o SqlServer é adicionado e removido pelo gerar-crud.sh só durante a geração.
POST na API respondia HTTP 500: InvalidCastException, “The field of type System.DateTime must be a string, array or ICollection type”.	Com --data-annotations, o Pomelo marca colunas datetime(6) com [MaxLength(6)] e o ASP.NET quebra ao validar.	O gerar.sh remove só esse atributo nos campos de data. Depois: POST 201.
Nomes errados: MotivoPerdum, Interacaos, Notificacaos.	O pluralizador padrão do EF só conhece inglês.	Um IPluralizer em português (Design/DesignTimeServices.cs). Exigiu incluir o asset “compile” do pacote Design no .csproj.
Regenerar do zero falhava: “Build failed”.	O scaffold compila o projeto antes de gerar, mas o Program.cs usa o MiniCrmContext, que acabou de ser apagado; o CRUD antigo também referenciava as entidades apagadas.	O gerar.sh apaga o CRUD antigo e cria um DbContext vazio provisório, que o scaffold sobrescreve.
A tela abria sem estilo (CSS com HTTP 500).	O dotnet run com --no-launch-profile sobe em Production, onde os arquivos estáticos do projeto não são servidos.	O rodar.sh define ASPNETCORE_ENVIRONMENT=Development (aplicação de demonstração).

Problema	Causa	Solução
15 avisos de compilação (CS8602) nas views geradas.	Com a nulabilidade ligada, as views geradas desreferenciam propriedades de navegação que o compilador considera possivelmente nulas.	Nullable desligado no .csproj: os avisos caíram de 15 para 1. Também evita um problema conhecido do scaffold com nulabilidade ligada (o ASP.NET tratar as navegações não anuláveis como campos obrigatórios), que não chegou a ser reproduzido aqui.
Repositório pesado: o wwwroot do template tinha 9,6 MB.	Sourcemaps do Bootstrap (24 arquivos, 6,2 MB) que só servem à depuração no navegador.	Removidos; o módulo ficou com ~3,4 MB no wwwroot.

3.8 Roteiro para a demonstração em aula

Qualquer integrante da equipe pode demonstrar a etapa seguindo estes passos, também descritos em dbfirst/README.md (os pré-requisitos de instalação do SDK estão lá):

```
cd dbfirst
docker compose up -d --wait   # 1. banco com as 9 tabelas
./scripts/gerar.sh            # 2. entidades e DbContext gerados do banco
./scripts/gerar-crud.sh       # 3. controllers, telas e API gerados
./scripts/rodar.sh            # 4. aplicação em http://localhost:8095
./scripts/demo-crud.sh        # 5. (outro terminal) CRUD pela API
./scripts/demo-telas.sh       # 6. CRUD pelos formulários
# navegador: http://localhost:8095  (telas)   http://localhost:8095/swagger  (API)
```

Para mostrar a inversão do fluxo ao vivo, crie uma tabela direto no MySQL e rode gerar.sh e gerar-crud.sh de novo: a tela da nova tabela aparece (veja o exemplo no README).

3.9 Limitações

- **Sem segurança:** a API e as telas geradas não têm login nem permissões. Servem para demonstrar a geração e não devem ser expostas; o sistema real é o backend principal.
- **Sem camada de service nem regras de negócio:** o CRUD gerado não aplica escopo de carteira, transições do funil nem status automático. Gera a estrutura; as regras continuam no backend.
- **Telas genéricas:** em inglês (Index, Create New), com campos na ordem alfabética das colunas, e não um frontend de negócio como o React do Mini CRM.
- **Tipos e nomes mecânicos:** bit(1) vira ulong (o campo excluido aparece como 0 ou 1) e a chave id_vendedor vira IdVendedorNavigation.

Anexo – Onde está cada entrega

Tudo está na branch nicolas/reuso-gru-dru-featureide-dbfirst do repositório MiniCRM-Reuso. O backend e o frontend existentes não foram alterados.

Item	Caminho
Este documento (Word e PDF)	docs/reuso/Entrega_Reuso_MiniCRM.docx e .pdf
Feature model (arquivo do FeatureIDE)	docs/reuso/featureide/model.xml
Configuração do produto atual	docs/reuso/featureide/configs/MiniCRM-atual.xml
Imagem exportada pelo FeatureIDE	docs/reuso/featureide/MiniCRM-FeatureModel.png
Prints do Eclipse/FeatureIDE	docs/reuso/featureide/prints/
Prints das telas geradas e saídas dos comandos	docs/reuso/dbfirst/prints/ e docs/reuso/dbfirst/saidas/
Projeto Database First em .NET (código gerado, scripts, Compose e dumps do banco)	dbfirst/ (veja dbfirst/README.md); banco em dbfirst/banco/